

BitPub documento di progettazione

1. Diagramma architetturale

Scelta del Pattern Architetturale: Microservizi Edge-Cloud

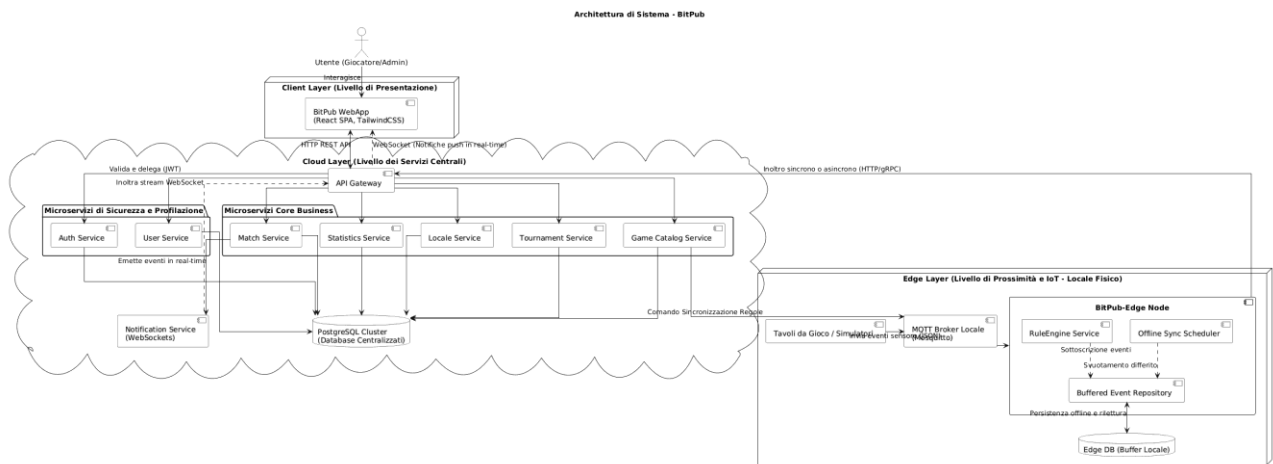
Per la progettazione di BitPub, il nostro gruppo ha scelto di adottare un pattern architetturale basato su Microservizi distribuiti su due livelli operativi principali: Edge e Cloud, coadiuvati da un'infrastruttura di comunicazione a Message Broker (MQTT). Il nostro intento principale è definire fin da subito una netta separazione tra l'acquisizione e validazione in tempo reale degli eventi fisici all'interno delle sale giochi e la gestione centralizzata della logica di business globale. In questa fase di design, riteniamo fondamentale disaccoppiare questi aspetti per garantire che lo svolgimento delle partite possa procedere in modo fluido e resiliente, indipendentemente dall'affidabilità della connessione internet del locale. Questa scelta strategica mira ad assicurare la massima tolleranza ai guasti e un'elevata scalabilità del sistema: il nostro obiettivo è facilitare la manutenzione futura del codice isolando i domini applicativi in servizi indipendenti.

Sulla base di questa analisi, abbiamo progettato l'architettura del sistema come segue:

Client Layer (Livello di Presentazione): Sarà il componente dedicato esclusivamente all'interfaccia grafica e all'interazione con l'utente (Giocatori e Amministratori). Intendiamo implementare questo livello utilizzando una WebApp Single Page Application (React), che definirà la struttura visiva e interattiva, consumando le API esposte dal backend e aggiornandosi dinamicamente tramite connessioni WebSocket per riflettere lo stato delle partite in tempo reale, senza incorporare logica di business complessa.

Cloud Layer (Livello dei Servizi Centrali): È progettato per fare da nucleo applicativo globale, accessibile tramite un API Gateway che funge da singolo punto di ingresso e vigila sulla sicurezza orchestrando l'autenticazione tramite JWT (gestita dall'Auth Service). Qui implementeremo microservizi specializzati (come il Match Service, il Tournament Service e lo Statistics Service), il cui compito sarà elaborare le richieste di sistema, gestire le classifiche, i tabelloni dei tornei e calcolare il consolidamento finale dei punteggi. La comunicazione tra questi servizi e i database avverrà tramite strati di accesso ai dati dedicati.

Edge Layer (Livello di Prossimità e IoT): Per mantenere reattivo il sistema e superare le limitazioni di rete dei locali fisici, è stato previsto l'inserimento di un Nodo Edge locale. L'intento è che questo componente raccolga gli eventi hardware dai tavoli da gioco tramite un broker MQTT locale (Mosquitto), li validi istantaneamente e agisca da buffer di sicurezza. In caso di disconnessione, il livello Edge si occuperà di conservare i dati per poi delegare a un processo di sincronizzazione asincrono l'aggiornamento del Cloud, isolando la logica di acquisizione da quella di persistenza globale.



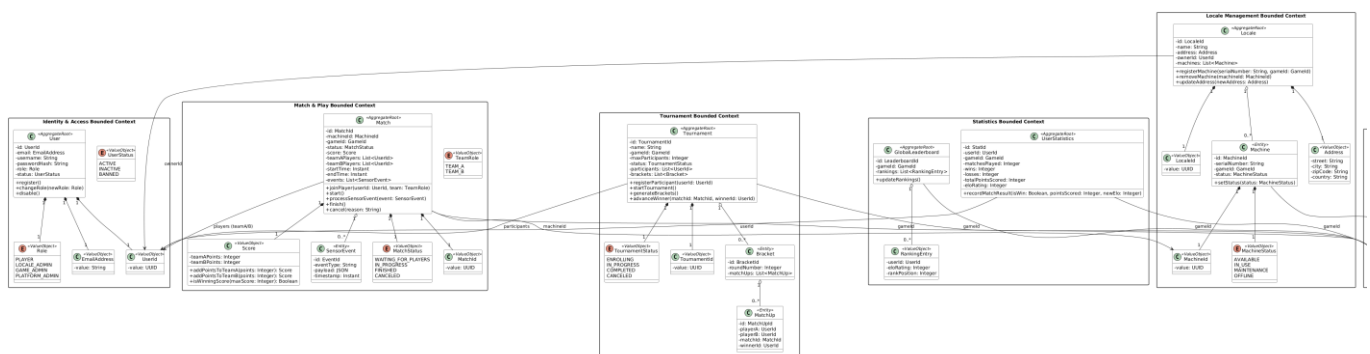
2. Diagramma delle classi di dominio

Modello di dominio

Il nostro modello di dominio è stato progettato per mappare accuratamente le entità fisiche del pub e le dinamiche logiche del gioco connesso. Come evidenziato nel diagramma delle classi di dominio, abbiamo definito le relazioni fondamentali tra gli utenti, gli spazi fisici, l'hardware e gli eventi di gioco, assicurando un tracciamento completo del ciclo di vita di ogni singola partita.

Classe	Attributi Principali	Responsabilità
User	id, username, email, password, role	Gestisce l'identità e i profili personali di Giocatori, Amministratori di Locale e di Piattaforma.
Locale	id, name, address, ownerId	Rappresenta la struttura fisica (es. il pub). Funge da contenitore organizzativo per le macchine da gioco.
GameInstance (Machine)	id, serialNumber, status, gameType, localeId	Rappresenta il tavolo o la singola postazione fisica vendibile/prenotabile all'interno di un locale.
GameType	id, name, rules	Contiene i metadati e le regole di chiusura e validazione per ciascuna tipologia di sport (es. Biliardo, Calciobalilla).

Classe	Attributi Principali	Responsabilità
Match	id, machinedId, startTime, endTime, status, currentScore	Associa i giocatori a una macchina in un lasso di tempo. Registra lo stato della competizione e il punteggio parziale/finale.
Tournament	id, name, gameType, startDate, endDate, maxParticipants	Rappresenta l'aggregazione di più partite ufficiali. Gestisce le iscrizioni degli utenti e la struttura del tabellone a eliminazione.
SensorEvent	eventId, matchId, timestamp, payload, eventType	Rappresenta il singolo impulso generato dall'hardware (es. un goal). Collega l'azione fisica allo stato temporale del match.



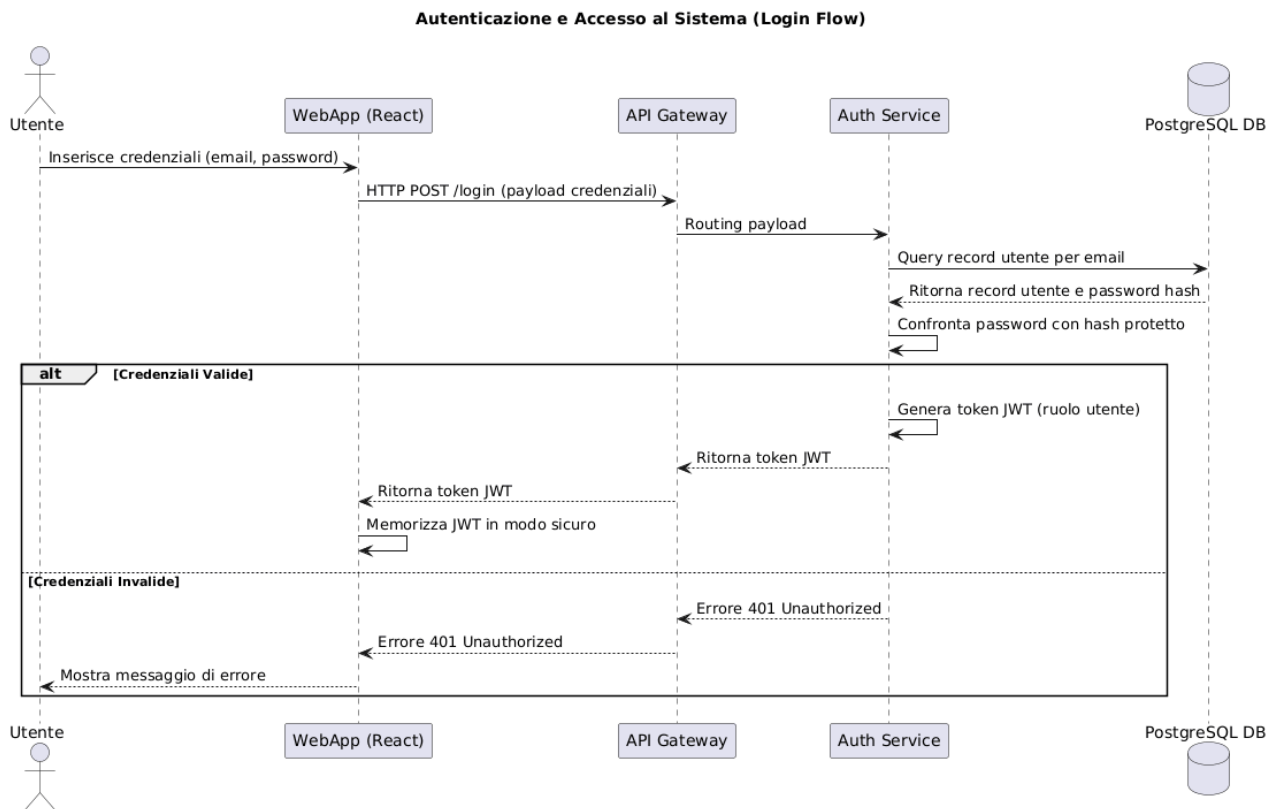
3. Diagrammi di sequenza

Al fine di documentare l'interazione dinamica e lo scambio di messaggi tra i diversi strati del sistema, abbiamo analizzato approfonditamente tutti i casi d'uso principali stabiliti dai requisiti di progetto, facendo esplicito riferimento ai relativi diagrammi di sequenza architetturali senza riprodurne la veste grafica.

Caso d'uso: Autenticazione e Accesso al Sistema (Login Flow)

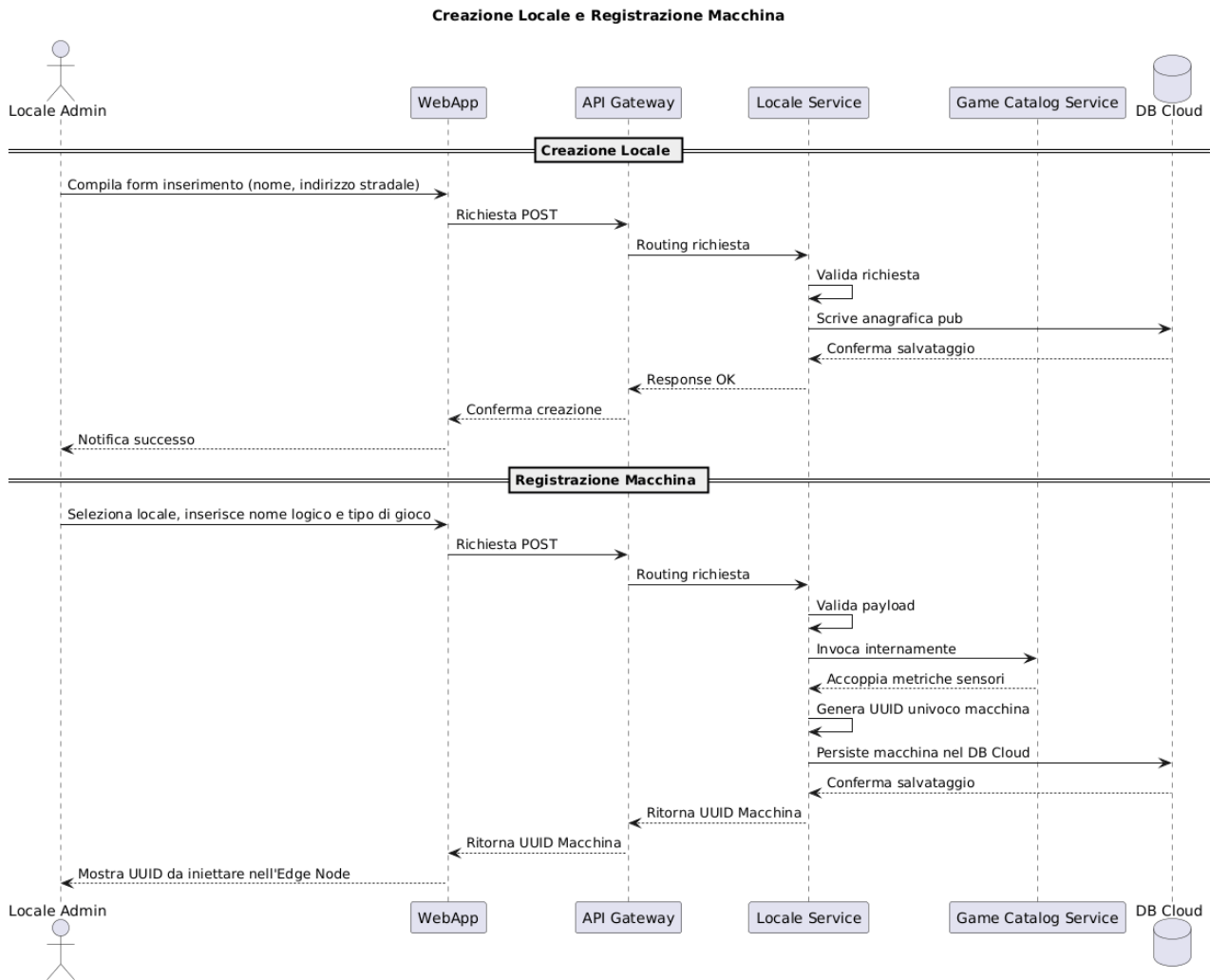
Lo scenario di autenticazione avvia la sessione operativa di qualsiasi attore che intenda interagire con le risorse protette della piattaforma. L'utente inserisce le proprie credenziali d'accesso (email e password) all'interno dell'interfaccia React della WebApp, la quale impacchetta i dati in una richiesta HTTP POST indirizzata all'endpoint pubblico dell'API

Gateway. Il Gateway, intercettando la chiamata, esegue il routing del payload verso l'Auth Service di backend. Quest'ultimo interroga il database PostgreSQL per verificare l'esistenza del record utente e confrontare la password immessa con l'hash protetto archiviato. In caso di corrispondenza e credenziali valide, l'Auth Service genera un token JWT firmato digitalmente contenente le informazioni sul ruolo dell'utente, restituendolo tramite il Gateway alla WebApp, che provvede a memorizzarlo in modo sicuro nel browser per allegarlo alle successive chiamate REST verso i microservizi di dominio.



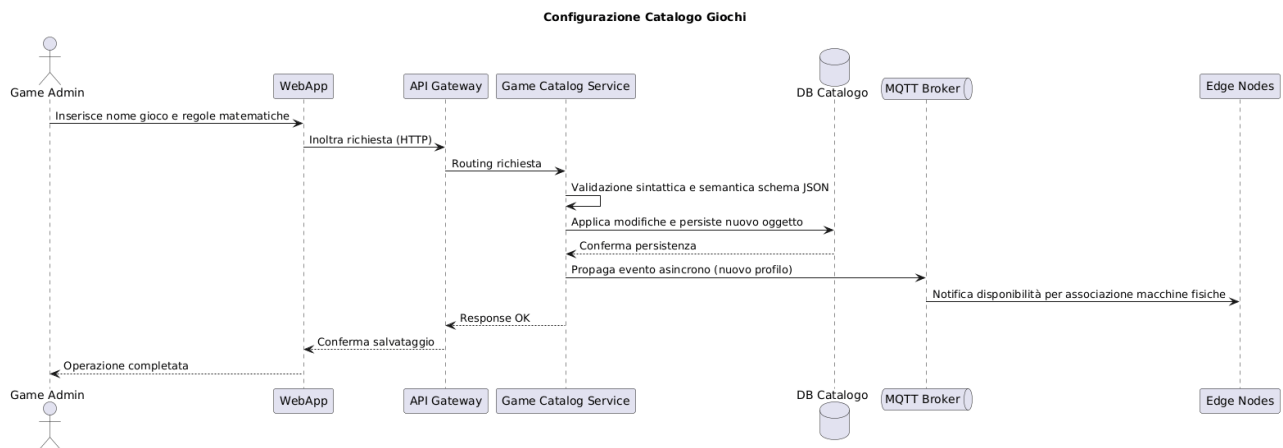
Caso d'uso 1: Creazione Locale e Registrazione Macchina

Questo processo consente agli utenti amministratori di censire le proprie strutture fisiche e predisporre l'identità dell'hardware IoT prima della sua installazione. L'Amministratore del Locale (Locale Admin), opportunamente autenticato, compila il form di inserimento inserendo il nome e l'indirizzo stradale del pub. La WebApp trasmette la richiesta al Locale Service passando attraverso i filtri di sicurezza dell'API Gateway. Il servizio valida la richiesta, scrive l'anagrafica del pub nel database cloud e restituisce la conferma al client. Successivamente, l'amministratore seleziona il locale creato e richiede la registrazione di una nuova macchina da gioco, specificandone il nome logico e il tipo di gioco associato attinto dal catalogo centrale. Il Locale Service, verificata la validità del payload, invoca internamente il Game Catalog Service per accoppiare le metriche dei sensori, genera un identificativo univoco UUID per la macchina e lo persiste nel DB Cloud, rendendolo disponibile alla WebApp affinché possa essere iniettato nel software dell'Edge Node periferico.



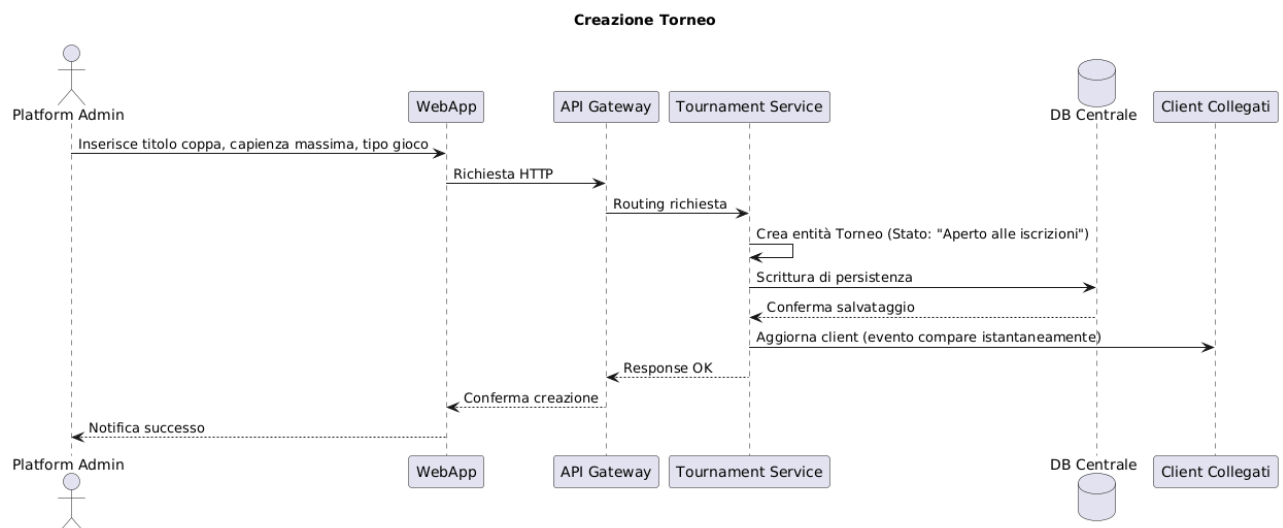
Caso d'uso 2: Configurazione Catalogo Giochi

L'interazione descrive l'inserimento di una nuova tipologia di sport o regolamento all'interno della piattaforma globale. L'Amministratore dei Giochi (Game Admin) accede alla sezione amministrativa del catalogo e definisce il payload configurativo comprendente il nome del gioco e le regole matematiche di chiusura dei match. Il client inoltra la richiesta al Game Catalog Service. Il microservizio esegue una validazione sintattica e semantica dello schema JSON delle regole, applica le modifiche persistendo il nuovo oggetto sul database del catalogo e propaga un evento asincrono via MQTT per notificare i nodi Edge della disponibilità del nuovo profilo di gioco, permettendone l'immediata associazione alle macchine fisiche operanti sul territorio.



Caso d'uso 3: Creazione Torneo

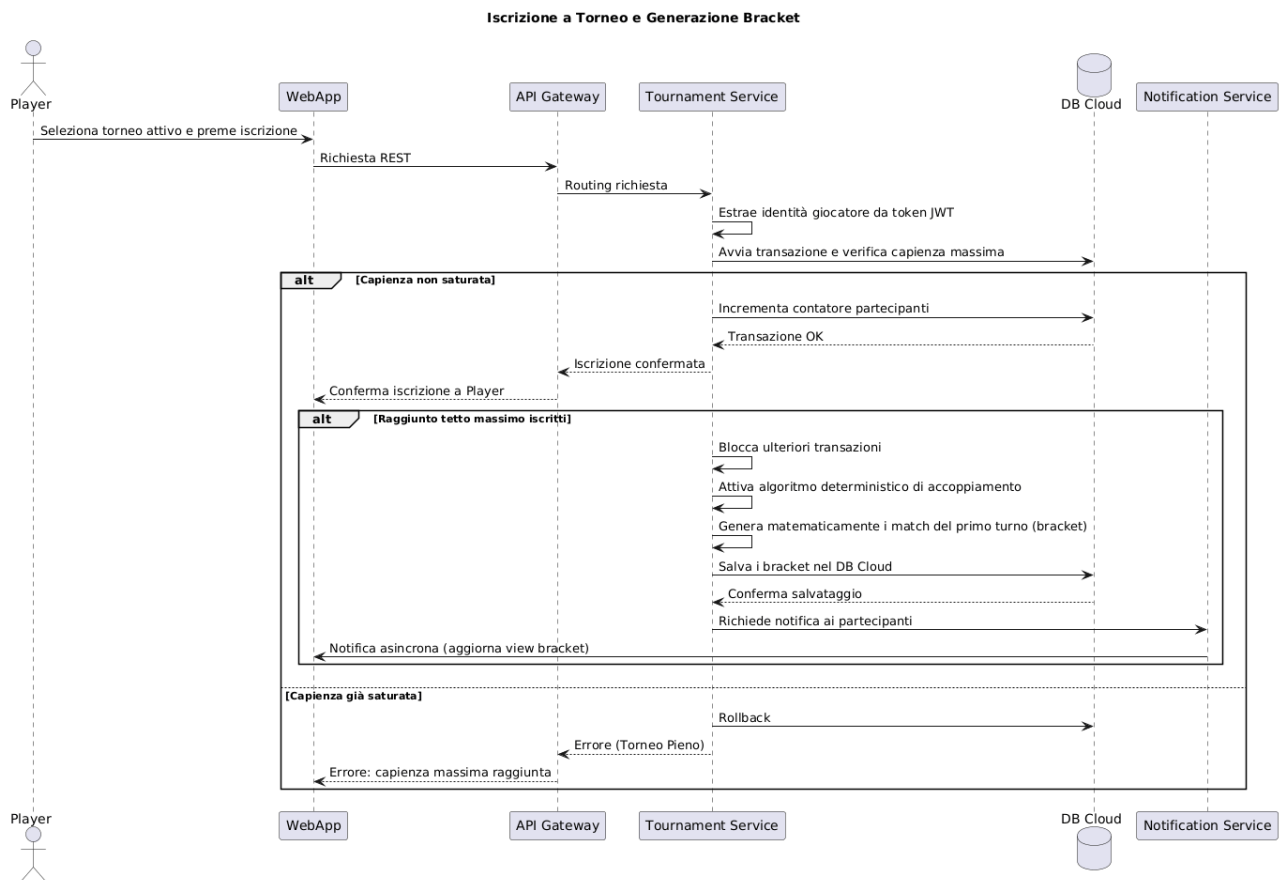
Questo scenario traccia la strutturazione di una competizione ufficiale accessibile dai giocatori remoti. L'Amministratore di Piattaforma (Platform Admin) interagisce con la dashboard dei tornei inserendo il titolo della coppa, la capienza massima del tabellone a eliminazione diretta e il tipo di gioco specifico richiesto. La richiesta attraversa il Gateway e viene presa in carico dal Tournament Service. Il servizio crea l'entità Torneo impostando lo stato iniziale su "Aperto alle iscrizioni", effettua la scrittura di persistenza sul database centrale e aggiorna i client collegati affinché l'evento compaia istantaneamente nella lista dei tornei consultabili dai player.



Caso d'uso 4: Iscrizione a Torneo e Generazione Bracket

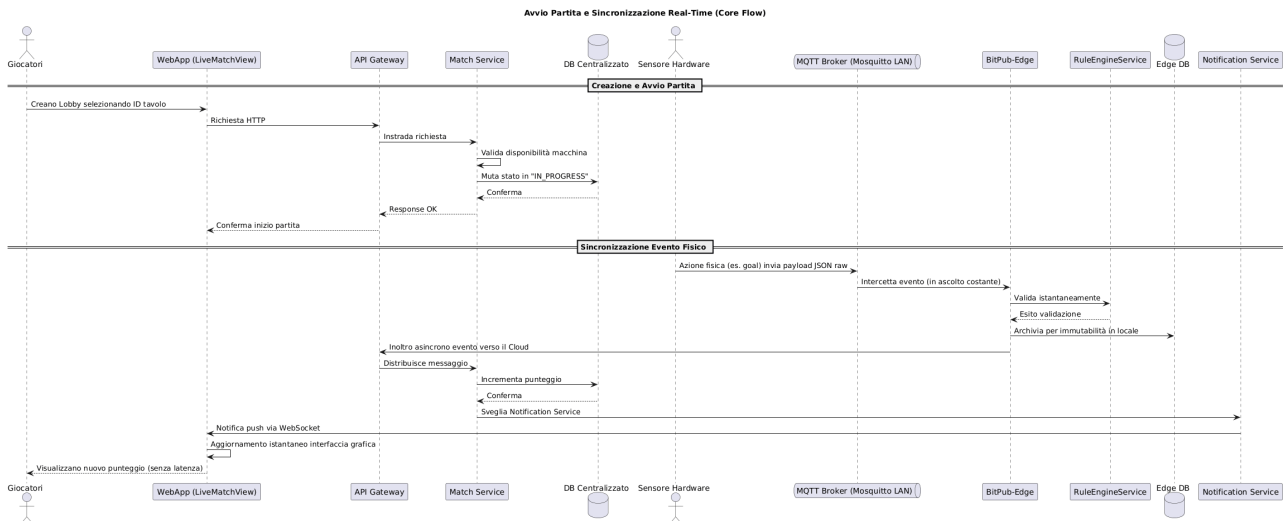
Il flusso esamina l'adesione di un utente a un torneo e il conseguente calcolo automatico degli abbinamenti competitivi. Il Giocatore (Player) naviga nella sezione tornei della WebApp e preme il comando di iscrizione in corrispondenza di una competizione attiva. Il Tournament Service riceve la richiesta via REST, estrae l'identità del giocatore dal token JWT e avvia una transazione sul database per incrementare il contatore dei partecipanti previa verifica che la capienza massima non sia stata ancora saturata. Qualora l'iscrizione confermata coincida con il raggiungimento del tetto massimo di iscritti previsto, il Tournament Service blocca le

ulteriori transazioni e attiva internamente un algoritmo deterministico di accoppiamento, generando matematicamente i match del primo turno (bracket), i quali vengono salvati nel DB Cloud e notificati asincronamente alle WebApp dei partecipanti coinvolti.



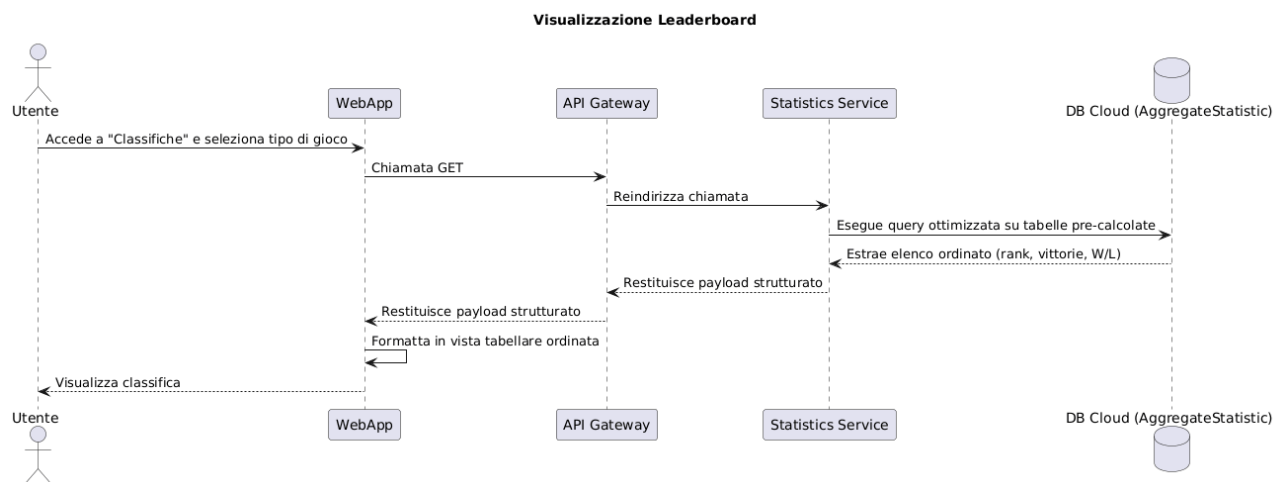
Caso d'uso 5: Avvio Partita e Sincronizzazione Real-Time (Core Flow)

Rappresenta l'interazione dinamica fondamentale del sistema, in cui cooperano attori umani, dispositivi IoT e servizi remoti. Due giocatori, fisicamente presenti davanti alla macchina da gioco nel pub, creano una Lobby selezionando l'ID del tavolo dalla WebApp. L'API Gateway instrada la richiesta al Match Service sul cloud, che valida la disponibilità della macchina e ne muta lo stato in "IN_PROGRESS". Quando la partita ha inizio, il verificarsi di un'azione fisica (es. un goal nel calciobalilla) fa scattare un sensore hardware, il quale invia un payload JSON raw al broker MQTT Mosquitto installato nella LAN del pub. Il microservizio BitPub-Edge, in ascolto costante sul broker locale, intercetta l'evento, lo valida istantaneamente tramite il RuleEngineService e lo archivia nell'Edge DB per garantire l'immutabilità del dato in locale. Subito dopo, l'Edge Node inoltra asincronamente l'evento validato verso l'API Gateway del Cloud. Il Gateway distribuisce il messaggio al Match Service, che incrementa il punteggio nel DB centralizzato e sveglia il Notification Service. Quest'ultimo invia una notifica push via WebSocket a tutte le WebApp collegate alla LiveMatchView, determinando l'aggiornamento istantaneo dell'interfaccia grafica dei client senza alcuna latenza percepita dall'utente.



Caso d'uso 6: Visualizzazione Leaderboard

L'interazione consente agli utenti di consultare i propri posizionamenti e lo storico competitivo. L'utente accede alla pagina "Classifiche" della WebApp e seleziona il tipo di gioco di interesse tramite un menu a tendina. Il client effettua una chiamata GET all'API Gateway, che la reindirizza allo Statistics Service. Per evitare pesanti scansioni sull'intera cronologia dei match globali, lo Statistics Service esegue una query ottimizzata su tabelle di aggregazione pre-calcolate (AggregateStatistic), estraendo l'elenco ordinato dei giocatori in base al rank, al numero di vittorie e al rapporto W/L, restituendo tempestivamente il payload strutturato alla UI React che provvede a formattarlo in una vista tabellare ordinata.

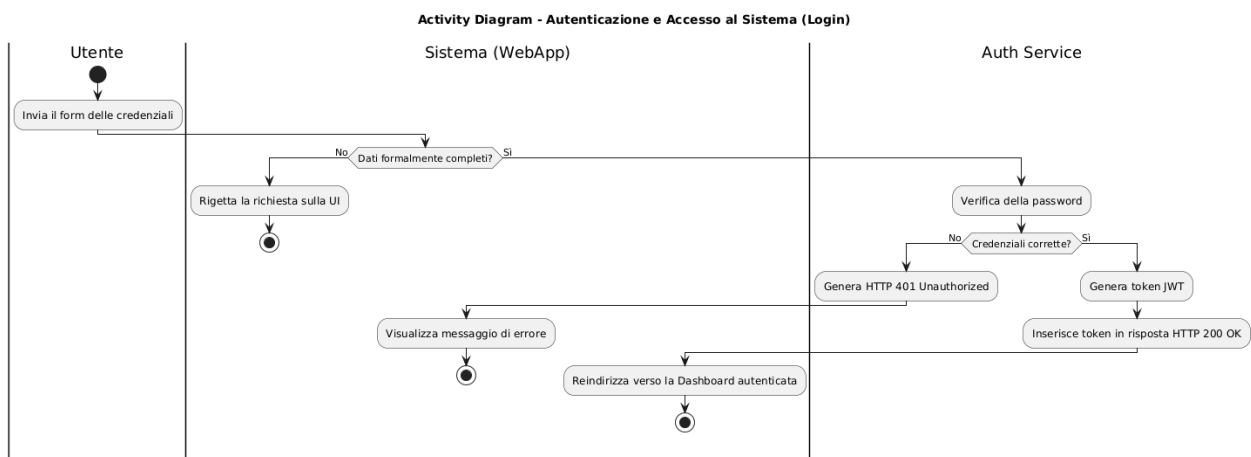


4. Diagrammi di attività

L'analisi dei flussi logici operativi e delle ramificazioni decisionali del sistema è stata formalizzata descrivendo il comportamento procedurale di ciascun processo, ponendo particolare enfasi sulla gestione delle eccezioni e sulla resilienza periferica stabilita nei requisiti di BitPub, facendo riferimento ai diagrammi di attività interni.

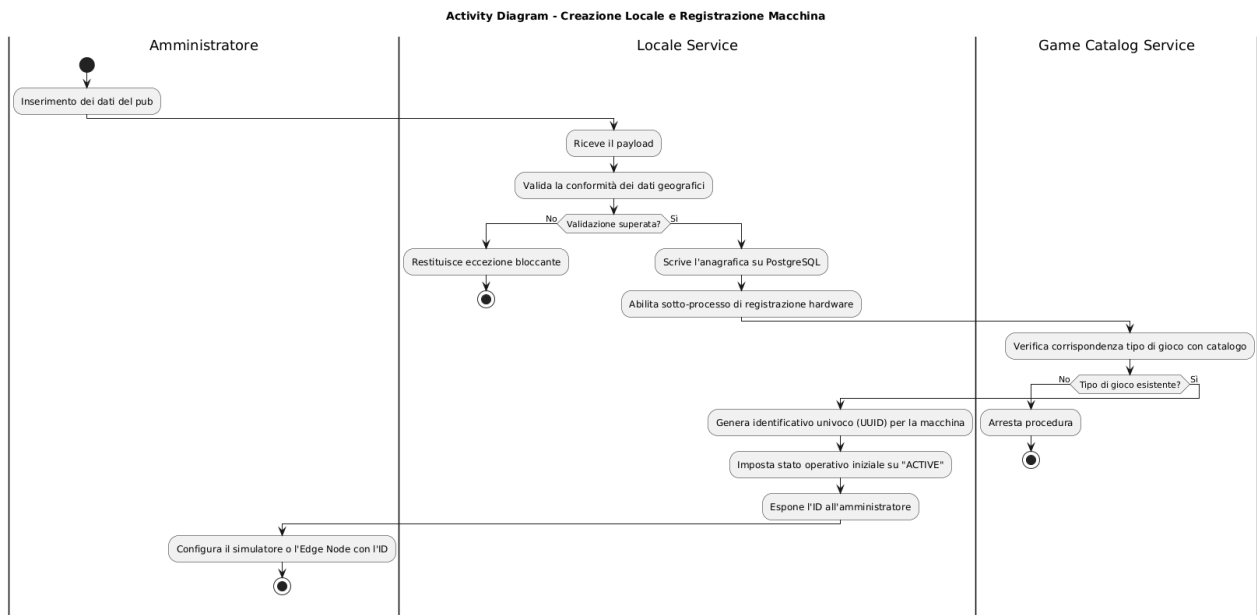
Caso d'uso: Autenticazione e Accesso al Sistema (Login)

Il flusso operativo inizia nel momento in cui l'utente invia il form delle credenziali. Il sistema esegue una prima decisione basata sulla presenza dei campi obbligatori. Se i dati sono formalmente incompleti, il processo si interrompe rigettando la richiesta sulla UI. In caso positivo, il controllo passa all'Auth Service che effettua la verifica della password. Se il controllo fallisce (credenziali errate), l'attività devia verso un ramo d'errore che genera un HTTP 401 Unauthorized, visualizzando un messaggio specifico sulla WebApp. Se le credenziali sono corrette, il sistema genera il token JWT, lo inserisce nella risposta di successo HTTP 200 OK e reindirizza il flusso utente verso la Dashboard autenticata.



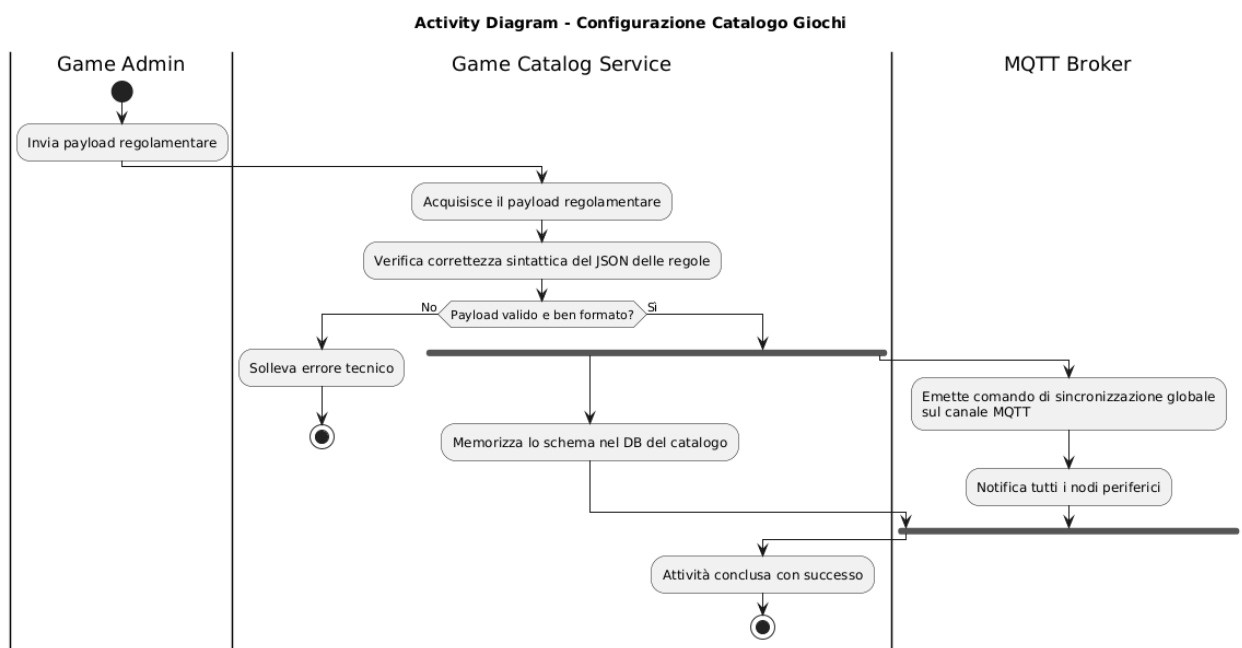
Caso d'uso 1: Creazione Locale e Registrazione Macchina

L'attività prende avvio con l'inserimento dei dati del pub da parte dell'amministratore. Il Locale Service riceve il payload e valida la conformità dei dati geografici. Se la validazione fallisce, viene restituita un'eccezione bloccante. In caso di successo, il sistema scrive l'anagrafica su PostgreSQL e abilita il sotto-processo di registrazione hardware. L'attività incontra un nodo decisionale in cui viene verificata la corrispondenza del tipo di gioco selezionato con il catalogo centrale: se il tipo di gioco è inesistente, la procedura si arresta; se il controllo ha esito positivo, il sistema genera l'identificativo univoco UUID per la macchina, imposta il suo stato operativo iniziale su "ACTIVE" e conclude il processo esponendo l'ID all'amministratore per la configurazione del simulatore o dell'Edge Node.



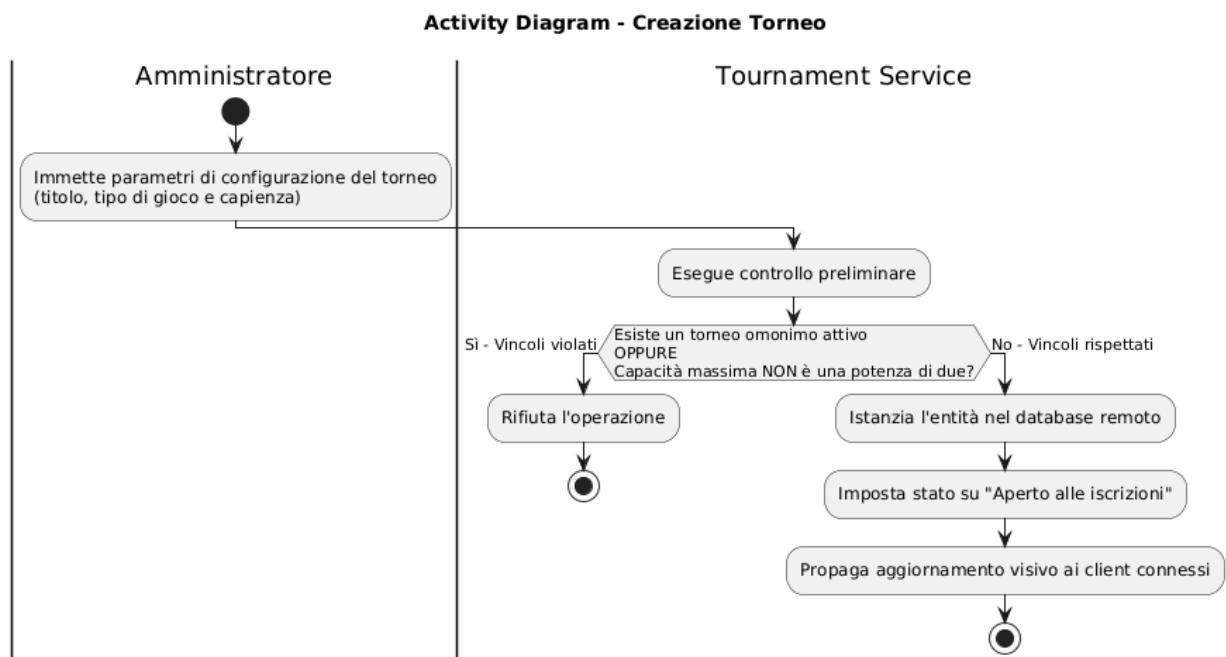
Caso d'uso 2: Configurazione Catalogo Giochi

Il flusso operativo del Game Catalog Service prevede l'acquisizione del payload regolamentare inviato dal Game Admin. Il sistema esegue una verifica formale sulla correttezza sintattica del JSON delle regole. Se il payload risulta invalido o malformato, l'attività solleva un errore tecnico interrompendo l'esecuzione; in caso di esito positivo, lo schema viene memorizzato nel DB del catalogo e viene parallelamente emesso un comando di sincronizzazione globale sul canale MQTT per notificare tutti i nodi periferici, concludendo l'attività con successo.



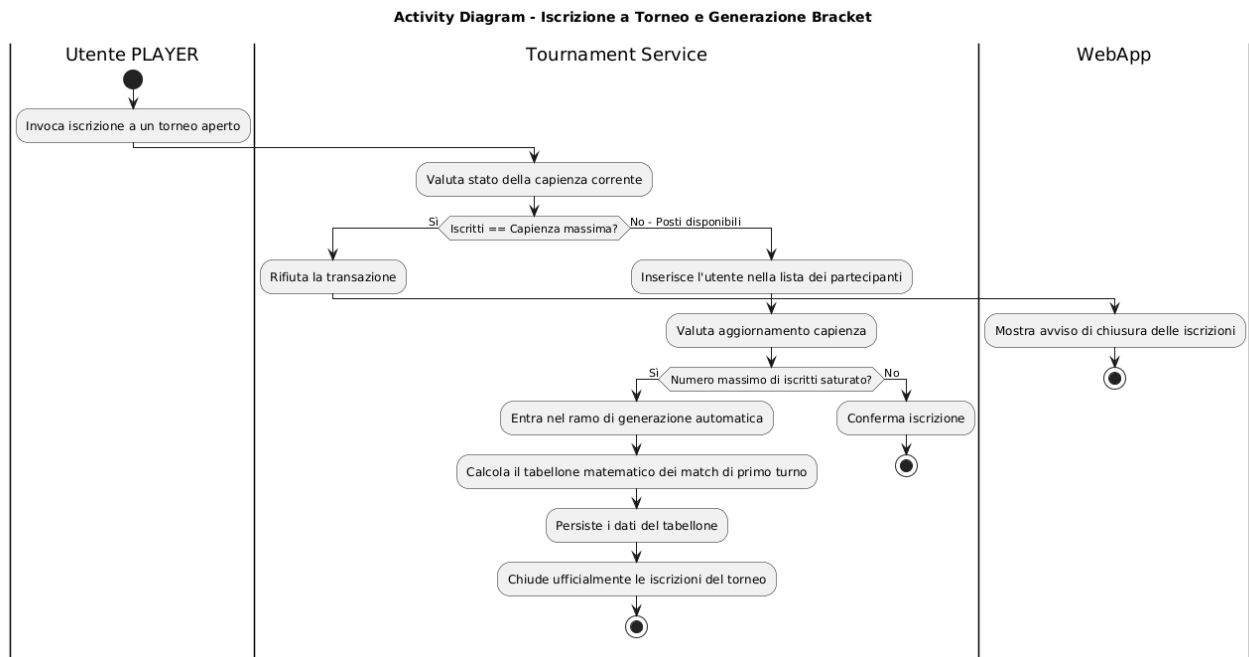
Caso d'uso 3: Creazione Torneo

La procedura amministrativa prende avvio con l'immissione dei parametri di configurazione del torneo (titolo, tipo di gioco e capienza). Il Tournament Service esegue un controllo preliminare per accertarsi che non esista un torneo omonimo attivo e che la capacità massima inserita sia una potenza di due adeguata a strutturare un tabellone ad eliminazione diretta. Se i vincoli non sono rispettati, l'operazione viene rifiutata; in caso contrario, il sistema istanzia l'entità nel database remoti impostando lo stato su "Aperto alle iscrizioni" e propaga l'aggiornamento visivo ai client connessi.



Caso d'uso 4: Iscrizione a Torneo e Generazione Bracket

L'attività si attiva quando un utente PLAYER invoca l'iscrizione a un torneo aperto. Il Tournament Service esegue una biforcazione decisionale critica valutando lo stato della capienza corrente (iscritti == capienza massima). Se la condizione è vera (Scenario A3 - Torneo al Completo), la transazione viene rifiutata, il flusso devia verso un ramo d'errore e viene mostrato l'avviso di chiusura delle iscrizioni, interrompendo il processo senza aggiungere l'utente. Se vi sono posti disponibili, l'utente viene inserito nella lista dei partecipanti. Successivamente, il sistema valuta una seconda condizione: se il numero massimo di iscritti è stato saturato proprio da quest'ultimo inserimento, l'attività entra nel ramo di generazione automatica, calcola il tabellone matematico dei match di primo turno, persiste i dati e chiude ufficialmente le iscrizioni del torneo.



Caso d'uso 5: Avvio Partita e Sincronizzazione Real-Time (Core Flow)

Questo complesso flusso operativo governa il ciclo vitale di una sessione di gioco connessa e le sue diramazioni di errore concorrenziale e di rete.

- Gestione Concorrenza (Scenario A2):** Nel momento in via due giocatori premono simultaneamente il tasto di avvio partita sulla medesima macchina, il Match Service intercetta le richieste concorrenziali ed elabora per prima quella arrivata al millisecondo precedente, mutando lo stato della macchina in "WAITING_FOR_PLAYERS". La seconda richiesta incontra un controllo decisionale che rileva lo stato variato, determinando il rifiuto della transazione con la notifica di errore "Macchina occupata" sulla UI del secondo utente.
- Ciclo degli Eventi Sensore e Resilienza Offline (Scenario A1):** Una volta che la partita è in stato "IN_PROGRESS", ogni impulso hardware genera un evento trasmesso al broker Mosquitto locale. Il BitPub-Edge riceve il messaggio JSON e attiva il RuleEngineService per convalidare la mossa in tempo reale. Subito dopo, il sistema valuta lo stato della connettività WAN verso il cloud server. Se la connessione è attiva, l'evento viene trasmesso immediatamente via HTTP/gRPC all'API Gateway cloud, che lo inoltra al Match Service per l'aggiornamento del DB centrale e la conseguente notifica push WebSocket verso la LiveMatchView.

Se il controllo rileva che la rete internet del pub è caduta (Scenario A1 - Disconnessione Cloud), l'attività devia verso un percorso resiliente protetto : il nodo Edge non scarta l'impulso sensore, ma invoca la classe BufferedEventRepository per salvare in modo sicuro il payload nel database locale Edge DB. L'attività di gioco prosegue localmente e l'Edge Node continua ad accumulare nel buffer tutti i punti successivi segnati al tavolo. Parallelamente, un processo in background regolato dall'OfflineSyncScheduler monitora lo stato della rete

[illegible]

L'attività prende avvio dalla richiesta esplicita di consultazione delle classifiche globali. Lo Statistics Service acquisisce i filtri sul tipo di gioco impostati dall'utente. Il sistema incontra un nodo di instradamento interno: interroga direttamente le tabelle AggregateStatistic per estrarre la classifica ordinata. Se il database risponde correttamente, la lista ordinata dei rank viene convertita in un payload DTO, trasmessa attraverso l'API Gateway e renderizzata graficamente nell'interfaccia tabellare React della WebApp, concludendo il processo in modalità totalmente stateless.

Activity Diagram - Visualizzazione Leaderboard

